

Deadline 6 — Quota Rollout & Admin Endpoints

Campus Resource Allocation System · Person A's column
Session 14 · 25 August 2026 · branch `main` · schema head `1ca8b85b7626`

What this session did

Took the quota mechanism that Deadline 4 proved on GPUs and applied it to the other two resources, then built the endpoints that configure it and the endpoint that reports it.

A's column is complete. **A'S COLUMN MET** B's column landed in session 13, so both are done — but Deadline 6 is **STILL OPEN** on its third line, the joint **swap review**, which needs both people.

No migration. `RoleQuota` shipped at Deadline 1; `alembic check` reports no drift. Second deadline running where A's usual opening move wasn't needed. The API grew from **21 to 26 routes**.

Files created

File	Lines	What it is
<code>scripts/check_quotas.py</code> NEW	576	The seventh gate. 61 assertions covering both new quotas, all four new endpoints, and the room-quota race over 20 trials.
<code>app/quotas/router.py</code> NEW	122	<code>GET/PUT /admin/quotas/{role}/{resource}</code> . Lives in <code>quotas/</code> rather than a new <code>admin/</code> package: the module that owns an invariant should own the endpoint that configures it.
<code>app/quotas/schemas.py</code> NEW	63	Policy and usage wire shapes, including the three distinct meanings of a quota value (a cap, zero, unlimited) plus the fourth that is a missing row.

Files modified

File	Δ	Change
app/quotas/service.py MOD	+188 / -2	Room and course helpers plus <code>usage_snapshot</code> . One mechanism, three resources — the only difference is the unit: GPUs SUM, rooms and courses COUNT.
app/rooms/service.py MOD	+105 / -28	The user-row lock, inserted exactly where Deadline 3 reserved space for it. <code>cancel_reservation</code> now takes it too, on the reservation's <i>owner</i> . Plus <code>update_room</code> .
scripts/check_rooms.py MOD	+119 / -3	B's gate, repaired — see the second finding below.
app/gpus/service.py MOD	+77	<code>update_cluster</code> and the <code>CapacityBelowAllocated</code> refusal.
app/users/router.py MOD	+47 / -3	<code>GET /me/quota</code> . It did <i>not</i> bring a <code>users/service.py</code> , contradicting that file's own Deadline 3 prediction — the domain logic belongs to <code>quotas/</code> .
app/rooms/router.py MOD	+49	<code>PATCH /rooms/{id}</code> and the two quota refusals.
app/gpus/router.py MOD	+40	<code>PATCH /gpus/{id}</code> .
app/courses/router.py MOD	+29 / -3	The fourth 409 on registration, and why it is <code>QUOTA_EXCEEDED</code> rather than a course-specific code.
app/courses/service.py MOD	+24	Course-load quota inside a lock Deadline 4 already held.
app/gpus/schemas.py · app/rooms/schemas.py · app/main.py	+53	PATCH bodies and the admin router mount.
4 documentation files	+438 / -11	<code>DECISIONS.md</code> (+250, the Deadline 6 section), <code>WORK_LOG.md</code> (+128, session 14), <code>EXECUTION_PLAN.md</code> , and <code>ARCHITECTURE_AND_WORKFLOWS.md</code> — where §13 needed correcting.

The result: the argument generalises, and now it is measured

Deadline 4 showed the quota lock mattered for GPUs. The room path gets its own number — one student at 1 of 2, firing two bookings at two *different* rooms simultaneously, 20 trials:

Build	Over-quota trials	Rooms held
resource lock only (no user lock)	19 / 20	{2:1, 3:19}
+ user-row lock	0 / 20	{2:20}

Compare Benchmark 2 at Deadline 4: **24/25 versus 0/25**. Same shape, same cause, third invariant.

The rooms case is arguably the *cleaner* demonstration, because rooms have no counter at all — the overlap invariant is enforced entirely by a GiST exclusion constraint, and that constraint is perfectly correct throughout. It simply cannot see a fact about the user. Two different rooms are two different resources; nothing serializes them but the user row.

Finding 1 — §13 describes a state the schema forbids

ARCHITECTURE_AND_WORKFLOWS.md §13 said an admin may lower a cluster "from 8 to 4 while 6 are allocated" and that allocated drains naturally. But `gpu_capacity_sane` is `allocated <= gpu_count` — Postgres rejects that row outright. Implemented literally, the documented behaviour reaches the caller as a **500 from a CHECK violation**.

The contradiction has been in the repo since Deadline 1. Nothing read the two against each other until an endpoint existed to try it.

Resolved by refusing the reduction, not by dropping the CHECK — `409 CAPACITY_BELOW_ALLOCATED`, decided against the row held `FOR UPDATE` so an allocation cannot commit between check and write.

The constraint is what makes a locking bug in the flagship transaction fail *loudly* instead of quietly overselling a cluster; `check_gpus.py` leans on it. Trading that for one sentence of documented convenience is a bad exchange. §13's intent survives exactly — nothing is ever evicted — and an admin shrinks to `allocated` now, further as holds release. Only the example is unavailable. §13 corrected.

Finding 2 — the change silently weakened B's strongest room assertion

`check_rooms.py` failed eleven assertions the moment the room quota landed: the seed student books ~10 slots to test intervals, and the cap is 2. That was expected — "*fix whatever B's benchmarks break*" is in this column.

The second effect was not expected and matters more. That script's strongest assertion fires eight simultaneous identical bookings at one room and requires exactly one row to land. All eight used the *same* student token.

Once `reserve_room` took the user-row lock, those eight serialized on that user's row and reached the exclusion constraint one at a time. **The assertion would still have passed** — while no longer measuring the constraint it is named for.

This project caught the identical bug at Deadline 4, in its own words: "*a capacity race in which every racer shares one account is not a capacity test.*" It arrived here from the opposite direction — not a test written wrong, but a correct test invalidated by a change somewhere else. Fixed with eight distinct racers.

Verification

`scripts/check_quotas.py` — **61/61 PASS**. Full regression run **twice**, to prove each gate leaves the state the next one assumes: all eight gates pass both times. B's three benchmarks re-run after the change: Benchmark 1 *PASS* — *exactly 10 in every trial*, Benchmarks 2 and 3 *PASS*. `alembic check` — no drift.

room 3 of 2	-> 409 QUOTA_EXCEEDED, nothing created
faculty, same slot	-> INTERVAL_CONFLICT, not QUOTA
me/quota while user row LOCKED	-> answers <- proves it takes no lock
(FACULTY,COURSE) via me/quota	-> configured:false, not unlimited
GET unseeded pair	-> 404, not a null limit
lower a quota under a holding	-> holds survive, next booking refused
course 2 of 1	-> QUOTA_EXCEEDED, not SCHEDULE_CONFLICT
drop then register the other	-> 201 <- DROPPED stops counting
block a room	-> RESOURCE_BLOCKED, hold NOT evicted
shrink GPU below allocated	-> 409 CAPACITY_BELOW_ALLOCATED, no change
shrink TO allocated	-> 200 <- boundary allowed
PATCH allocated directly	-> ignored, still 2

Decisions worth being able to defend

- Gate order on registration:** `ALREADY_ENROLLED` → quota → `SCHEDULE_CONFLICT` → `CAPACITY_EXHAUSTED`. Quota comes after already-enrolled because the count includes the seat being asked about; before the other two

because a student at their limit cannot register for anything.

- **GET /me/quota takes no lock**, and the test proves it rather than asserting about it — the gate holds the user row `FOR UPDATE` in a second session and the endpoint still answers within 5s.
- **The read endpoint must not fail closed the way the transaction does.** `limit_for` raising on a missing policy is right inside an allocation and wrong in a dashboard: `(FACULTY, COURSE)` is deliberately unseeded, so a faculty member asking their limits would get a 409 for a reasonable question. Reported as `configured: false` instead.
- **PUT upserts**, because the URL identifies the pair rather than a row id — and it is the only in-band way to fix a missing policy that would otherwise 409 every faculty registration.
- **Room quota is not time-aware**, deliberately: nothing in this system expires reservations, so `ACTIVE` is the definition of held. Making the quota time-aware would invent a third state the constraint, the cancel path and the availability endpoint all know nothing about.

What is left

- **The swap review is the only thing between here and Deadline 6.** One hour, both people: A walks B through the GPU transaction, B walks A through the exclusion constraint and the harness.
- **session.py still needs A's review** — carried from sessions 12 and 13, a shared file changed solo twice. It should have been the first thing done this session and was not.
- **Items 7, 9, 10, 11 all block Deadline 7** and are untouched. Item 9 (global lock order vs. waitlist promotion) is A's transaction and now more expensive: **three** paths take user→resource, so promotion contradicting it costs more than it did.
- Benchmark 4 (waitlist) still does not exist — Deadline 7.